

C++ & SQLite Task Tracker CLI

Practical Exercise: Prepared Statements, Parameter Binding & Iterative Data Access

1. Overview & Objectives

The objective of this assignment is to implement a command-line interface (CLI) application in C++ using the native **SQLite3 C API**. You will practice safe parameter binding using prepared statements (`?` placeholders), row-by-row query retrieval, and resource cleanup.

This foundational structure is designed to be refactored into clean Architectural Patterns (Entities & DAOs) in future phases.

2. Database Schema

Your application should create and connect to a local database file named `tasks.db`.

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for each task.
title	TEXT	NOT NULL	Task description entered by the user.
status	TEXT	NOT NULL	Current state (defaults to <code>"PENDING"</code>).

```
CREATE TABLE IF NOT EXISTS tasks (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    status TEXT NOT NULL  
);
```

3. Implementation Requirements

A. Table Initialization

- On program execution, open (or create) `tasks.db`.
- Execute the table creation SQL via `sqlite3_exec()`.

B. Interactive Insertion Loop

- Prepare the insert SQL query once using `sqlite3_prepare_v2()`:
`INSERT INTO tasks (title, status) VALUES (?, ?);`

- Prompt the user for a task title using `std::getline(std::cin, title)`.
- If the user submits a non-empty string:
 - Bind `title` to parameter index 1 and `"PENDING"` to parameter index 2 using `sqlite3_bind_text()` with `SQLITE_TRANSIENT`.
 - Execute the statement with `sqlite3_step()` and verify the return code is `SQLITE_DONE` (101).
 - Reset state and clear bindings using `sqlite3_reset()` and `sqlite3_clear_bindings()`.
- Terminate the loop when the user presses Enter on an empty line.
- Finalize the prepared statement using `sqlite3_finalize()`.

C. Query & Output Display

- Prepare a select query: `SELECT id, title, status FROM tasks;`
- Loop through the results using `sqlite3_step() == SQLITE_ROW`.
- Retrieve column values with `sqlite3_column_int()` and `sqlite3_column_text()`.
- Print all task records formatted to standard output (`std::cout`).
- Finalize statement and close the database connection (`sqlite3_close()`).

4. Future Refactoring Target (Phase 2 Roadmap)

After completing the procedural implementation above, prepare to refactor the codebase into an Object-Oriented DAO architecture:

Target Architecture:

1. **Task Entity Class:** Encapsulates `id`, `title`, and `status` fields with getters/setters.
2. **TaskDAO Class:** Contains data manipulation methods:
 - `bool insert(const Task& task)`
 - `std::vector<Task> getAll()`